Date.now()

📄 Título

Date.now() – El timestamp actual en milisegundos

📝 Definición

`Date.now()` es un método **estático** de JavaScript que devuelve el **número de milisegundos** transcurridos desde el **1 de enero de 1970, 00:00:00 UTC** (Unix Epoch) hasta el momento actual.

🧠 ¿Qué es?

Es la forma más eficiente y directa de obtener el **timestamp actual** en JavaScript.

A diferencia de `new Date().getTime()`, `Date.now()` es un **método estático** que no requiere crear una instancia de `Date`, lo que lo hace más rápido y con menos overhead.

Características clave:

- Devuelve un **número entero** (precisión de milisegundos).
- Es **independiente de la zona horaria** (siempre en UTC).
- Es la base para medir **tiempos de ejecución** y **rendimiento**.
- Se usa comúnmente para generar **identificadores únicos** basados en tiempo.
- Es la forma recomendada de obtener el timestamp actual.

🎯 ¿Para qué sirve?

- Obtener el **timestamp actual** de forma eficiente.
- Medir **tiempos de ejecución** de código.
- Generar **identificadores únicos** y **nombres de archivo**.
- Implementar **cachés con expiración**.

- Crear **logs** y **marcas de tiempo**.
- Sincronizar **eventos y temporizadores**.
- Calcular **diferencias de tiempo**.
- Generar **semillas** para números aleatorios.
- Implementar **rate limiting** y **throttling**.

⚙️ ¿Cómo funciona?

`Date.now()` es un método estático que consulta el reloj del sistema y devuelve el timestamp actual en milisegundos.

Detalles técnicos:

- El Epoch es el 1 de enero de 1970, 00:00:00 UTC.
- El valor devuelto es un número entero de 64 bits.
- Es equivalente a `new Date().getTime()` pero más eficiente.
- En navegadores modernos, la precisión está limitada a milisegundos.
- Para mediciones de alta precisión, usar `performance.now()`.

📝 Sintaxis / Estructura (JavaScript)

```
Date.now()
```

Retorna:

- Un número que representa los milisegundos desde el Epoch hasta el momento actual.

Errores:

- Nunca lanza errores (siempre devuelve un número válido).

💡 Ejemplos

```
// 1. Uso básico
const timestamp = Date.now();
console.log(timestamp); // 1703539200000 (ejemplo)

// 2. Comparación con new Date().getTime()
console.log(Date.now() === new Date().getTime()); // true
(casi siempre)

// 3. Medir tiempo de ejecución
const start = Date.now();

// Simular operación costosa
for (let i = 0; i < 1000000; i++) {
  Math.sqrt(i);
}

const end = Date.now();
console.log(`Tiempo de ejecución:${end - start}ms`);

// 4. Generar identificador único
function generateId() {
  return `${Date.now()}-${Math.random().toString(36).substr
(2, 6)}`;
}

console.log(generateId()); // "1703539200000-abc123"

// 5. Timestamp en segundos (Unix)
function getUnixTimestamp() {
  return Math.floor(Date.now() / 1000);
}

console.log(getUnixTimestamp()); // 1703539200

// 6. Timestamp para nombre de archivo
function getTimestampFileName() {
  const now = new Date(Date.now());
  const year = now.getFullYear();
```

```
  const month = String(now.getMonth() + 1).padStart(2,
'0');
  const day = String(now.getDate()).padStart(2, '0');
  const hours = String(now.getHours()).padStart(2, '0');
  const minutes = String(now.getMinutes()).padStart(2,
'0');
  const seconds = String(now.getSeconds()).padStart(2,
'0');

  return `${year}${month}${day}_${hours}${minutes}${second
s}`;
}

console.log(getTimestampFileName()); // "20241225_153045"

// 7. Cache con expiración
class TimedCache {
  constructor(ttl = 60000) { // 1 minuto por defecto
    this.cache = new Map();
    this.ttl = ttl;
  }

  set(key, value) {
    this.cache.set(key, {
      value,
      timestamp: Date.now()
    });
  }

  get(key) {
    const entry = this.cache.get(key);
    if (!entry) return null;

    if (Date.now() - entry.timestamp > this.ttl) {
      this.cache.delete(key);
      return null;
    }
```

```
    return entry.value;
  }

  cleanup() {
    const now = Date.now();
    for (const [key, entry] of this.cache) {
      if (now - entry.timestamp > this.ttl) {
        this.cache.delete(key);
      }
    }
  }
}

const cache = new TimedCache(5000);
cache.set('key', 'value');
console.log(cache.get('key')); // "value"

// 8. Rate limiting
class RateLimiter {
  constructor(limit = 10, windowMs = 60000) {
    this.limit = limit;
    this.windowMs = windowMs;
    this.requests = [];
  }

  isAllowed() {
    const now = Date.now();

    // Eliminar solicitudes antiguas
    this.requests = this.requests.filter(
      timestamp => now - timestamp < this.windowMs
    );

    if (this.requests.length >= this.limit) {
      return false;
    }

    this.requests.push(now);
```

```
    return true;
  }

  getRemaining() {
    const now = Date.now();
    this.requests = this.requests.filter(
      timestamp => now - timestamp < this.windowMs
    );
    return this.limit - this.requests.length;
  }
}

const limiter = new RateLimiter(5, 10000);
console.log(limiter.isAllowed()); // true
console.log(limiter.isAllowed()); // true
console.log(limiter.isAllowed()); // true
console.log(limiter.isAllowed()); // true
console.log(limiter.isAllowed()); // true
console.log(limiter.isAllowed()); // false

// 9. Timestamp para logs
function log(message, level = 'info') {
  const timestamp = new Date(Date.now()).toISOString();
  console.log(`[${timestamp}] [${level.toUpperCase()}]${mes
sage}`);
}

log('Aplicación iniciada', 'info');
log('Error al conectar', 'error');

// 10. Comparación de performance
function measurePerformance(fn, iterations = 1000000) {
  const start = Date.now();
  for (let i = 0; i < iterations; i++) {
    fn();
  }
  const end = Date.now();
  return end - start;
```

```
}

const timeNewDate = measurePerformance(() => new Date().get
Time());
const timeDateNow = measurePerformance(() => Date.now());

console.log(`new Date().getTime():${timeNewDate}ms`);
console.log(`Date.now():${timeDateNow}ms`);
// Date.now() es más rápido

// 11. Semilla para números aleatorios
function seededRandom(seed = Date.now()) {
  return function() {
    seed = (seed * 9301 + 49297) % 233280;
    return seed / 233280;
  };
}

const random = seededRandom();
console.log(random()); // Número pseudo-aleatorio basado en
timestamp

// 12. Días desde el Epoch
function daysSinceEpoch() {
  return Math.floor(Date.now() / (1000 * 60 * 60 * 24));
}

console.log(daysSinceEpoch()); // Días desde 1970-01-01

// 13. Milisegundos hasta una fecha específica
function msUntil(date) {
  const target = new Date(date).getTime();
  const now = Date.now();
  return Math.max(0, target - now);
}

console.log(msUntil('2024-12-31T23:59:59Z')); // Milisegund
os hasta Año Nuevo
```

```
// 14. Verificar si es el mismo día
function isSameDay(date1, date2) {
  const d1 = new Date(date1);
  const d2 = new Date(date2);
  return d1.getFullYear() === d2.getFullYear() &&
         d1.getMonth() === d2.getMonth() &&
         d1.getDate() === d2.getDate();
}

console.log(isSameDay(Date.now(), Date.now())); // true

// 15. Generar rango de timestamps
function generateTimestampRange(intervalMinutes = 60, count
= 24) {
  const now = Date.now();
  const interval = intervalMinutes * 60 * 1000;
  const timestamps = [];

  for (let i = 0; i < count; i++) {
    timestamps.push(now - (count - 1 - i) * interval);
  }

  return timestamps;
}

console.log(generateTimestampRange(60, 24)); // Últimas 24
horas
```

⚠️ Errores comunes

- **Usar `new Date().getTime()` en lugar de `Date.now()`:**

```
// ❌ MENOS EFICIENTE
const now = new Date().getTime();
```

```
// ✅ MÁS EFICIENTE
const now = Date.now();
```

- **Pensar que devuelve segundos**:

```
// ❌ MAL - Devuelve milisegundos
const timestamp = Date.now(); // 1703539200000

// ✅ BIEN - Para segundos
const unixTimestamp = Math.floor(Date.now() / 1000); //
1703539200
```

- **Usar `Date.now()` para mediciones de precisión ultra alta**:

```
// ❌ NO recomendado para mediciones de alta precisión
const start = Date.now();
// operación rápida
const duration = Date.now() - start; // Puede ser 0ms

// ✅ Usar performance.now() para mediciones de alta pre
cisión
const startPerf = performance.now();
// operación rápida
const durationPerf = performance.now() - startPerf; // P
recisión de microsegundos
```

- **Asumir que los timestamps son únicos**:

```
// ❌ Pueden colisionar en sistemas muy rápidos
const id1 = Date.now();
const id2 = Date.now();

// ✅ Mejor combinación
```

```
const id = `${Date.now()}-${Math.random().toString(36).substr(2, 6)}`;
```

💼 Casos de uso

1. **Sistema de autenticación con tokens**:

```
class TokenManager {
  constructor(expirationMs = 3600000) {
    this.tokens = new Map();
    this.expirationMs = expirationMs;
  }

  createToken(userId) {
    const token = this.generateToken();
    const now = Date.now();

    this.tokens.set(token, {
      userId,
      createdAt: now,
      expiresAt: now + this.expirationMs
    });

    return token;
  }

  generateToken() {
    return Buffer.from(
      `${Date.now()}-${Math.random().toString(36).substr(2, 8)}`
    ).toString('base64');
  }

  validateToken(token) {
    const entry = this.tokens.get(token);
    if (!entry) return null;
```

```
    if (Date.now() > entry.expiresAt) {
      this.tokens.delete(token);
      return null;
    }

    return entry.userId;
  }

  refreshToken(token) {
    const entry = this.tokens.get(token);
    if (!entry) return null;

    entry.expiresAt = Date.now() + this.expirationMs;
    return entry.expiresAt;
  }

  cleanup() {
    const now = Date.now();
    for (const [token, entry] of this.tokens) {
      if (now > entry.expiresAt) {
        this.tokens.delete(token);
      }
    }
  }
}

const tokenManager = new TokenManager(60000);
const token = tokenManager.createToken('user123');
console.log(tokenManager.validateToken(token)); // "user
123"
```

2. **Sistema de caché con TTL**:

```
class MemoryCache {
  constructor(defaultTTL = 60000) {
    this.cache = new Map();
    this.defaultTTL = defaultTTL;
```

```
  }

  set(key, value, ttl = this.defaultTTL) {
    this.cache.set(key, {
      value,
      expires: Date.now() + ttl
    });
  }

  get(key) {
    const entry = this.cache.get(key);
    if (!entry) return null;

    if (Date.now() > entry.expires) {
      this.cache.delete(key);
      return null;
    }

    return entry.value;
  }

  getRemainingTTL(key) {
    const entry = this.cache.get(key);
    if (!entry) return -1;

    const remaining = entry.expires - Date.now();
    return Math.max(0, remaining);
  }

  cleanup() {
    const now = Date.now();
    for (const [key, entry] of this.cache) {
      if (now > entry.expires) {
        this.cache.delete(key);
      }
    }
  }
```

```
  getStats() {
    const now = Date.now();
    let total = 0;
    let expired = 0;

    for (const entry of this.cache.values()) {
      total++;
      if (now > entry.expires) expired++;
    }

    return {
      total,
      expired,
      active: total - expired
    };
  }
}

const cache = new MemoryCache(5000);
cache.set('key', 'value');
console.log(cache.get('key')); // "value"
console.log(cache.getRemainingTTL('key')); // ~5000ms
```

3. **Generador de IDs secuenciales**:

```
class IDGenerator {
  constructor(prefix = 'id') {
    this.prefix = prefix;
    this.counter = 0;
    this.lastTimestamp = 0;
  }

  generate() {
    const now = Date.now();

    // Si el timestamp ha cambiado, resetear el contador
    if (now !== this.lastTimestamp) {
```

```
      this.counter = 0;
      this.lastTimestamp = now;
    }

    this.counter++;

    return `${this.prefix}_${now}_${this.counter.toStrin
g().padStart(4, '0')}`;
  }

  // Generar ID con formato personalizado
  generateCustom(format = 'timestamp') {
    switch(format) {
      case 'timestamp':
        return `${this.prefix}_${Date.now()}`;
      case 'sequential':
        return `${this.prefix}_${Date.now()}_${this.coun
ter++}`;
      case 'short':
        return `${this.prefix}_${Date.now().toString(3
6)}`;
      case 'mixed':
        return `${this.prefix}_${Date.now().toString(3
6)}_${Math.random().toString(36).substr(2, 4)}`;
      default:
        return this.generate();
    }
  }
}

const generator = new IDGenerator('user');
console.log(generator.generate()); // "user_170353920000
0_0001"
console.log(generator.generateCustom('short')); // "user
_2m9l7k"
```

4. **Sistema de métricas y análisis**:

```
class MetricsCollector {
  constructor() {
    this.metrics = [];
    this.sessionStart = Date.now();
  }

  recordEvent(name, value = null) {
    this.metrics.push({
      name,
      value,
      timestamp: Date.now(),
      elapsed: Date.now() - this.sessionStart
    });
  }

  getTiming(name) {
    const events = this.metrics.filter(m => m.name === n
ame);
    if (events.length < 2) return null;

    const first = events[0];
    const last = events[events.length - 1];

    return {
      start: first.timestamp,
      end: last.timestamp,
      duration: last.timestamp - first.timestamp,
      count: events.length
    };
  }

  getAverageTime(name) {
    const events = this.metrics.filter(m => m.name === n
ame);
    if (events.length < 2) return 0;

    const timings = [];
    for (let i = 1; i < events.length; i++) {
```

```
      timings.push(events[i].timestamp - events[i-1].tim
estamp);
    }

    return timings.reduce((a, b) => a + b, 0) / timings.
length;
  }

  getSessionDuration() {
    return Date.now() - this.sessionStart;
  }

  getMetricsByTimeRange(startTime, endTime) {
    return this.metrics.filter(m =>
      m.timestamp >= startTime && m.timestamp <= endTime
    );
  }
}

const metrics = new MetricsCollector();
metrics.recordEvent('page_load', 100);
setTimeout(() => metrics.recordEvent('api_call', 50), 10
0);
setTimeout(() => metrics.recordEvent('page_load', 80), 2
00);
console.log(metrics.getTiming('page_load'));
console.log(metrics.getSessionDuration());
```

5. **Sistema de logs con timestamp**:

```
class Logger {
  constructor(level = 'info') {
    this.level = level;
    this.logs = [];
    this.maxLogs = 1000;
  }
```

```
  log(message, level = 'info', data = null) {
    const timestamp = Date.now();
    const entry = {
      timestamp,
      isoDate: new Date(timestamp).toISOString(),
      level,
      message,
      data
    };

    this.logs.push(entry);

    if (this.logs.length > this.maxLogs) {
      this.logs = this.logs.slice(-this.maxLogs);
    }

    // Output en consola con formato
    console.log(`[${entry.isoDate}] [${level.toUpperCase
()}]${message}`);

    return entry;
  }

  info(message, data = null) {
    return this.log(message, 'info', data);
  }

  warn(message, data = null) {
    return this.log(message, 'warn', data);
  }

  error(message, data = null) {
    return this.log(message, 'error', data);
  }

  getLogsByTimeRange(startTime, endTime) {
    return this.logs.filter(l =>
      l.timestamp >= startTime && l.timestamp <= endTime
```

```
    );
  }

  getLogsByLevel(level) {
    return this.logs.filter(l => l.level === level);
  }

  getRecentLogs(seconds = 60) {
    const cutoff = Date.now() - seconds * 1000;
    return this.logs.filter(l => l.timestamp >= cutoff);
  }

  getStats() {
    const stats = {
      total: this.logs.length,
      levels: {},
      timeRange: {
        oldest: this.logs.length > 0 ? this.logs[0].time
stamp : null,
        newest: this.logs.length > 0 ? this.logs[this.lo
gs.length - 1].timestamp : null
      }
    };

    this.logs.forEach(log => {
      stats.levels[log.level] = (stats.levels[log.level]
|| 0) + 1;
    });

    return stats;
  }
}

const logger = new Logger();
logger.info('Aplicación iniciada');
logger.warn('Memoria baja', { usage: '75%' });
logger.error('Error al procesar', { code: 500 });
console.log(logger.getStats());
```

6. **Sistema de notificaciones programadas**:

```
class ScheduledNotifier {
  constructor() {
    this.notifications = [];
    this.interval = null;
  }

  schedule(message, delayMs) {
    const scheduledTime = Date.now() + delayMs;

    this.notifications.push({
      id: Date.now() + '-' + Math.random().toString(36).
substr(2, 6),
      message,
      scheduledTime,
      triggered: false
    });

    this.notifications.sort((a, b) => a.scheduledTime -
b.scheduledTime);
    this.startChecking();

    return this.notifications[this.notifications.length
- 1];
  }

  startChecking() {
    if (this.interval) return;

    this.interval = setInterval(() => {
      const now = Date.now();
      const toTrigger = this.notifications.filter(n =>
        !n.triggered && n.scheduledTime <= now
      );

      toTrigger.forEach(n => {
```

```
        this.trigger(n);
        n.triggered = true;
      });

      // Limpiar notificaciones antiguas
      this.notifications = this.notifications.filter(
        n => !n.triggered || n.scheduledTime > now - 864
00000
      );
    }, 1000);
  }

  trigger(notification) {
    console.log(`🔔${notification.message}`);
    // Aquí se podría mostrar una notificación real
  }

  getPending() {
    const now = Date.now();
    return this.notifications
      .filter(n => !n.triggered)
      .map(n => ({
        ...n,
        remaining: Math.max(0, n.scheduledTime - now),
        remainingFormatted: this.formatTime(Math.max(0,
n.scheduledTime - now))
      }));
  }

  formatTime(ms) {
    if (ms <= 0) return 'Ahora';
    const seconds = Math.floor(ms / 1000);
    if (seconds < 60) return `${seconds}s`;
    const minutes = Math.floor(seconds / 60);
    if (minutes < 60) return `${minutes}m${seconds % 60}
s`;
    const hours = Math.floor(minutes / 60);
    if (hours < 24) return `${hours}h${minutes % 60}m`;
```

```
    const days = Math.floor(hours / 24);
    return `${days}d${hours % 24}h`;
  }

  stop() {
    if (this.interval) {
      clearInterval(this.interval);
      this.interval = null;
    }
  }
}

const notifier = new ScheduledNotifier();
notifier.schedule('Recordatorio en 5 segundos', 5000);
notifier.schedule('Recordatorio en 10 segundos', 10000);
console.log(notifier.getPending());
```

🧩 Relación con otros conceptos

- `Date.prototype.getTime()`: Similar, pero requiere instancia de Date.
- `performance.now()`: Timestamp de alta precisión para mediciones.
- `process.hrtime()`: Timestamp de alta precisión en Node.js.
- `Date.parse()`: Convierte string de fecha a timestamp.
- `Date.UTC()`: Crea timestamp para fecha UTC específica.
- `new Date(timestamp)`: Crea fecha desde timestamp.

🧠 Analogía

Imagina que `Date.now()` es como un **cronómetro universal** que ha estado corriendo desde la medianoche del 1 de enero de 1970. Cada vez que lo miras, te dice exactamente cuántos milisegundos han pasado desde ese momento. Es la forma más rápida y directa de saber la hora actual en JavaScript.

📚 Fuente

- MDN Web Docs – Date.now()

- ECMAScript Specification – Date.now
- MDN – Date